IF69D-S11 e OPEET014-OPEL - Processamento Digital de Imagens

Aula 04

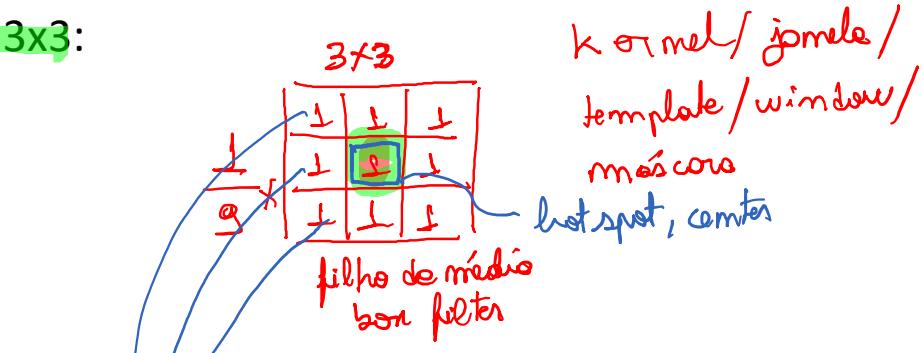
Processamento baseado em vizinhança

Universidade Tecnológica Federal do Paraná - UTFPR
Departamento Acadêmico de Eletrônica – DAELN-CT
Prof. Gustavo B. Borba e Prof. Humberto R. Gamba

15.set.2026

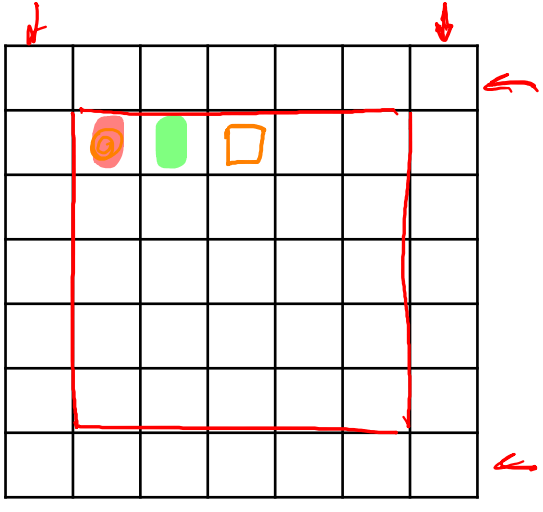
Processamento baseado em vizinhança (neighborhood processing)

Vizinhança 3x3:



	1	2	3	4	5	6	7
1	10	10	10	10	10	10	10
	10	10	10	10	10	10	10
	30	30	30	30	30	30	30
	30	30	30	30	30	30	30
	50	50	50	50	50	50	50
	50	50	50	50	50	50	50
7	70	70	70	70	70	70	70

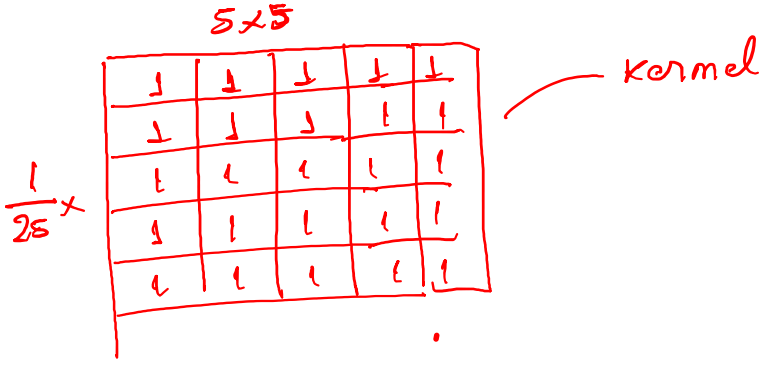
Input image 7x7



output image 7x7

Processamento baseado em vizinhança (neighborhood processing)

Vizinhança 5x5:



10	10	10	10	10	10	10
10	10	10	10	10	10	10
30	30	30	30	30	30	30
30	30	30	30	30	30	30
50	50	50	50	50	50	50
50	50	50	50	50	50	50
70	70	70	70	70	70	70

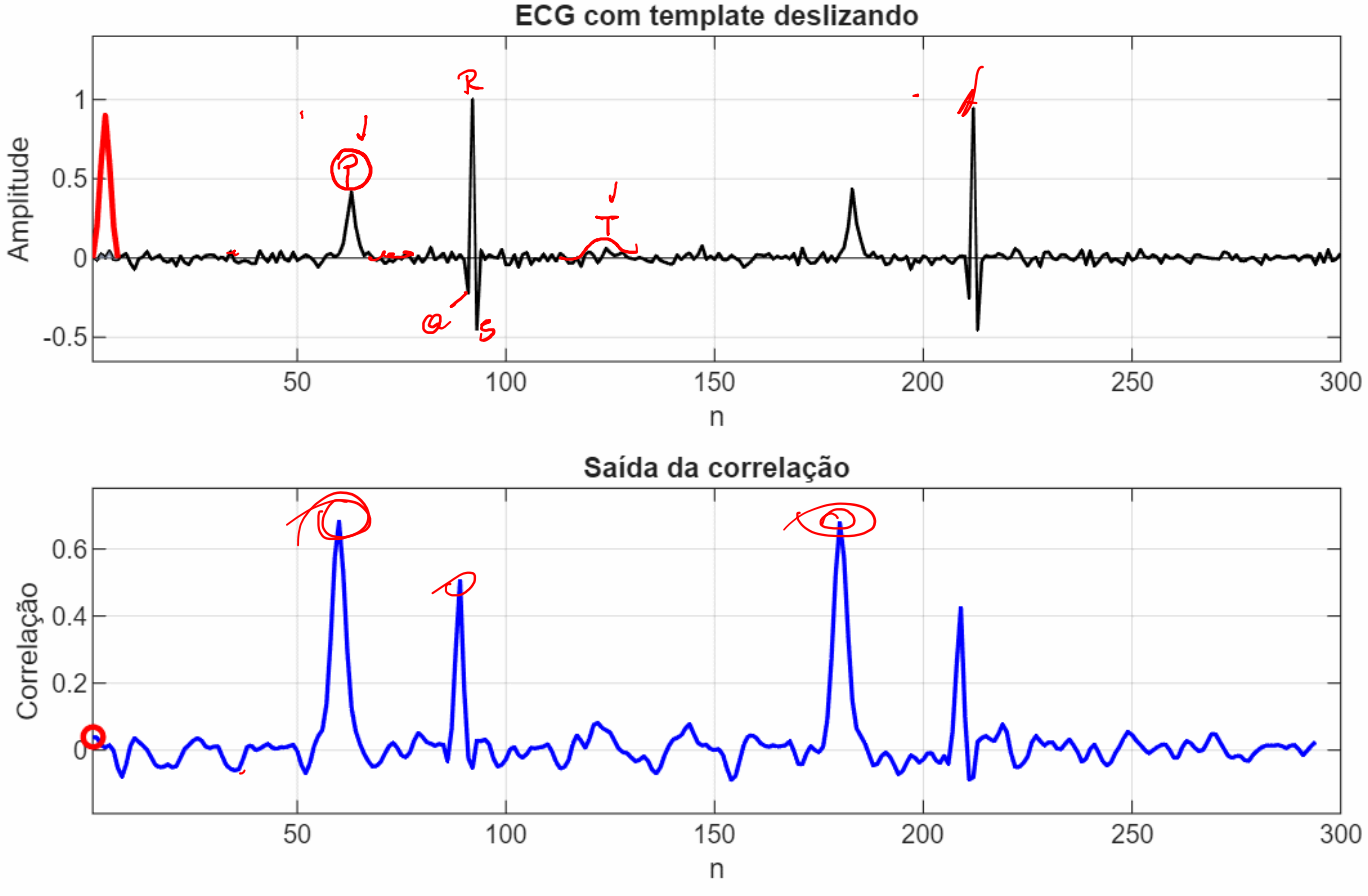
Input image 7x7

output image 7x7

filtrada

Correlação: Exemplo de detecção da onda P de uma eletrocardiograma

$$c(k) = \sum f(k)\ g(n+k)$$



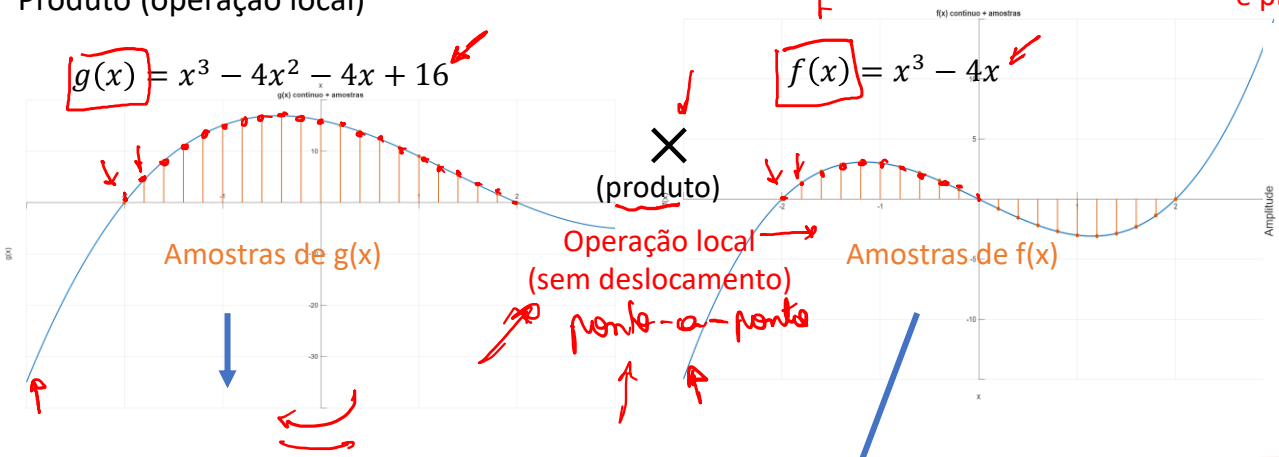
% Ex_Correcao_ECG_GIF_Final.m

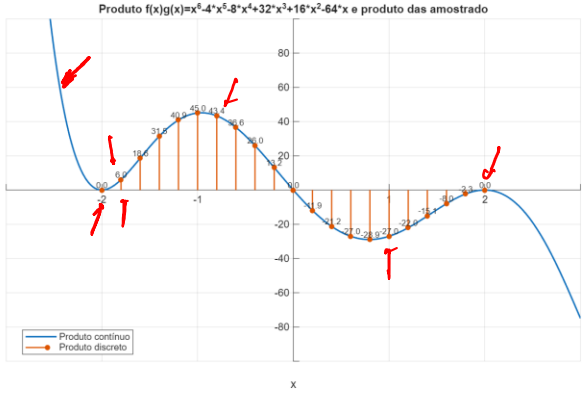
Convolução: Convolução vs produto ponto a ponto

f(x)g(x)=x^6-4*x^5-8*x^4+32*x^3+16*x^2-64*x
e produto das amostrado (ponto a ponto)

Produto (operação local)

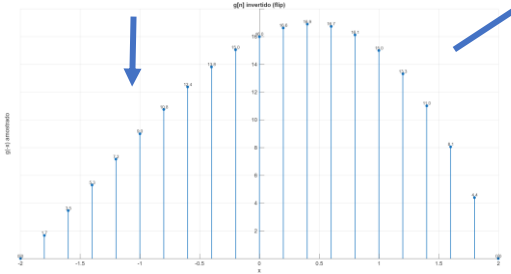
$g(x) = x^3 - 4x^2 - 4x + 16$ × (produto) $f(x) = x^3 - 4x$





Convolução (operação com deslocamento)

flip das amostras de g(x), permite implementar o termo $g(n-k)$ da convolução.



* (convolução)

Quanto de um sinal combina com o outro enquanto desliza um sobre o outro.

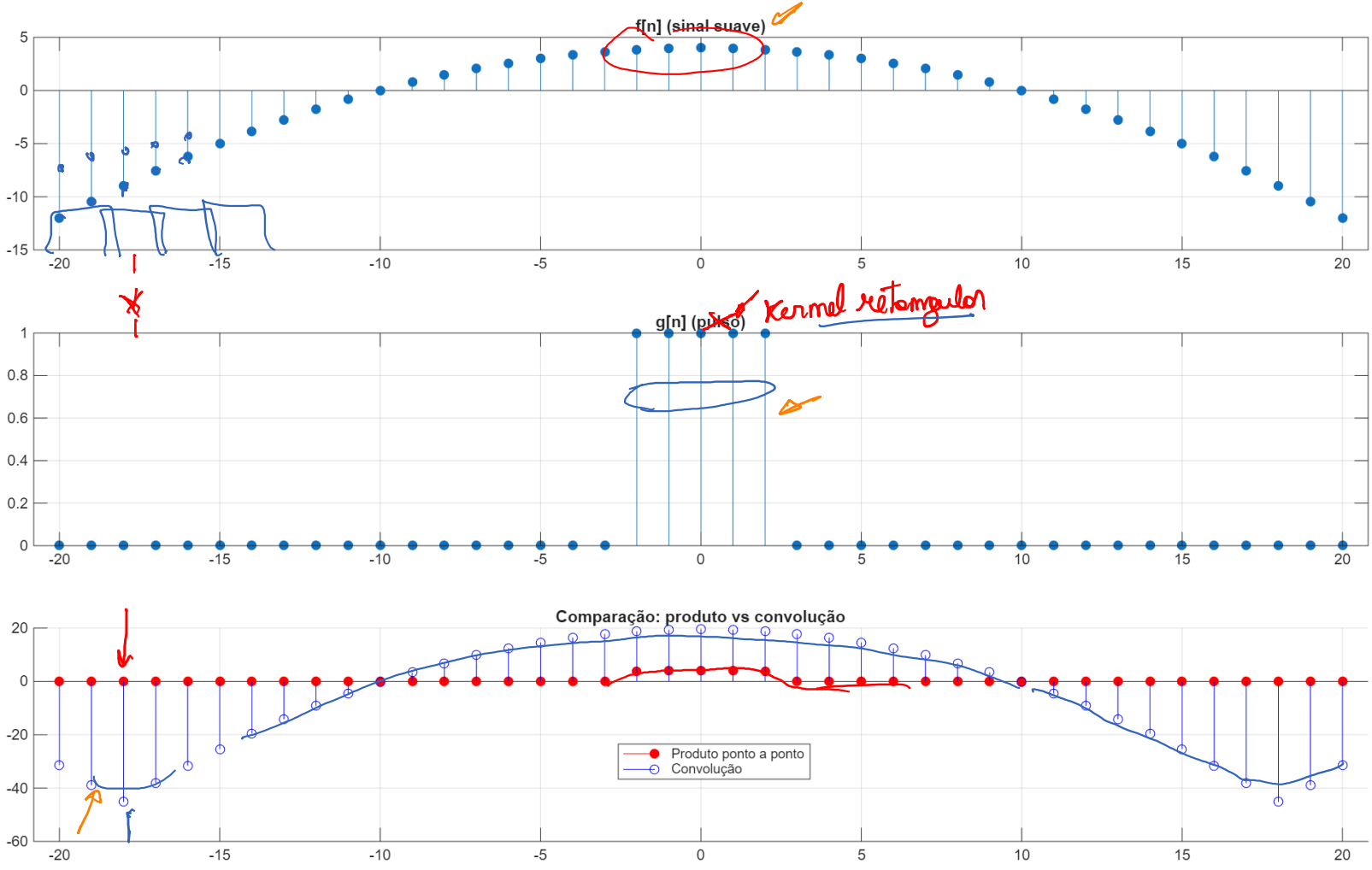
$$c[n] = \sum_k f(k)g(n-k)$$

MATLAB
c = conv(f, g)



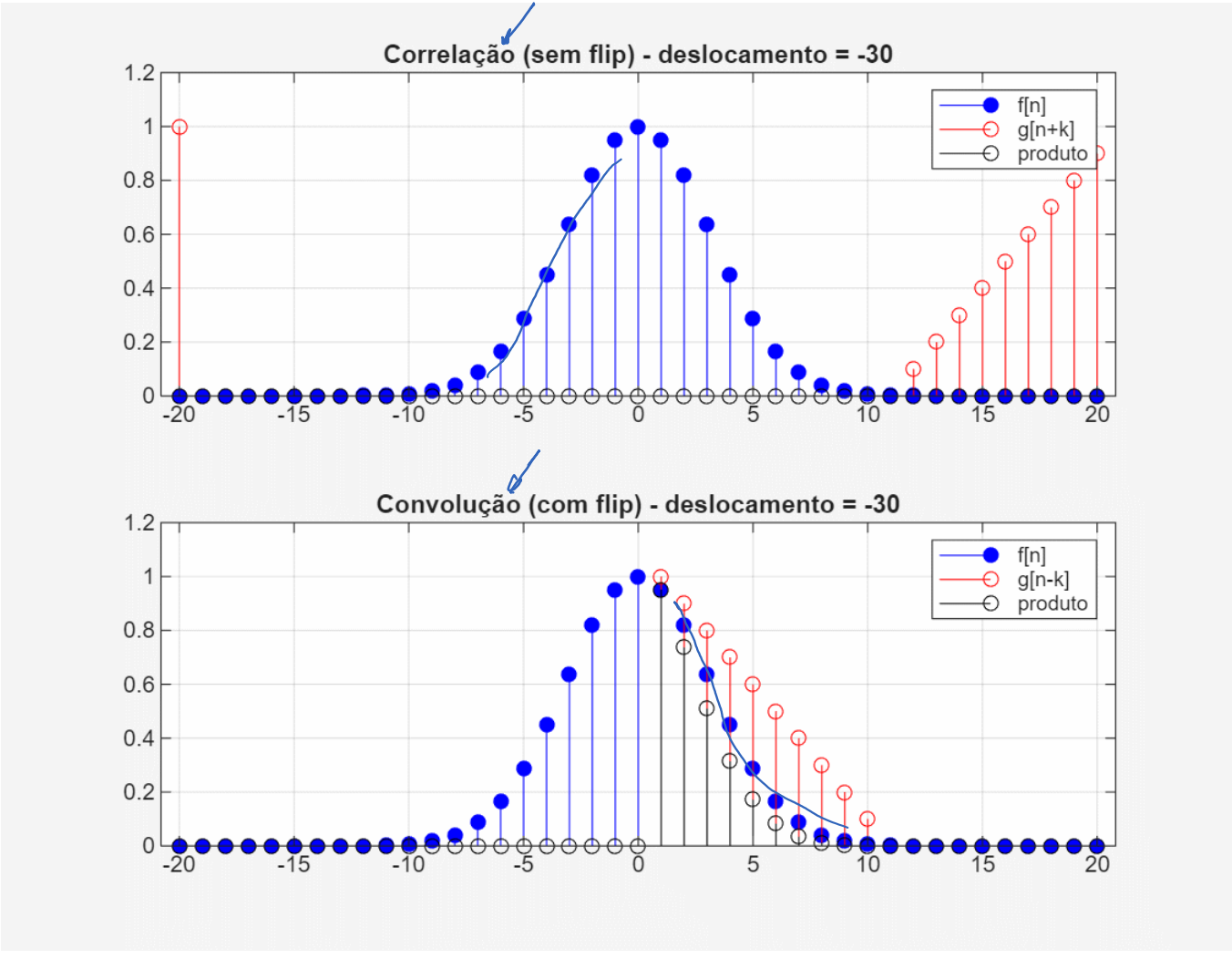
Ex_Conv_Prod_2_Polinomios.m

Convolução: Convolução vs produto ponto a ponto



Ex_2_Conv_Com_Impulso.m

Convolução: Convolução vs Correlação



Neighborhood Processing: Filtro espacial

Convolução passo-a-passo

$I' = I * H$

$$I'(u,v) = \sum I(u-i, v-j) \cdot H(i,j)$$

$$= \sum I(u+i, v+j) \cdot H(-i,-j)$$

formalização matemática do processo de convolução

↳ porque ocorre o flip da região da imagem de entrada I

$u = 2 \quad v = 2$

$$I(2-(-1), 2-(-1)) \cdot H(-1,-1)$$

$$I(3, 3) \cdot H(-1,-1)$$

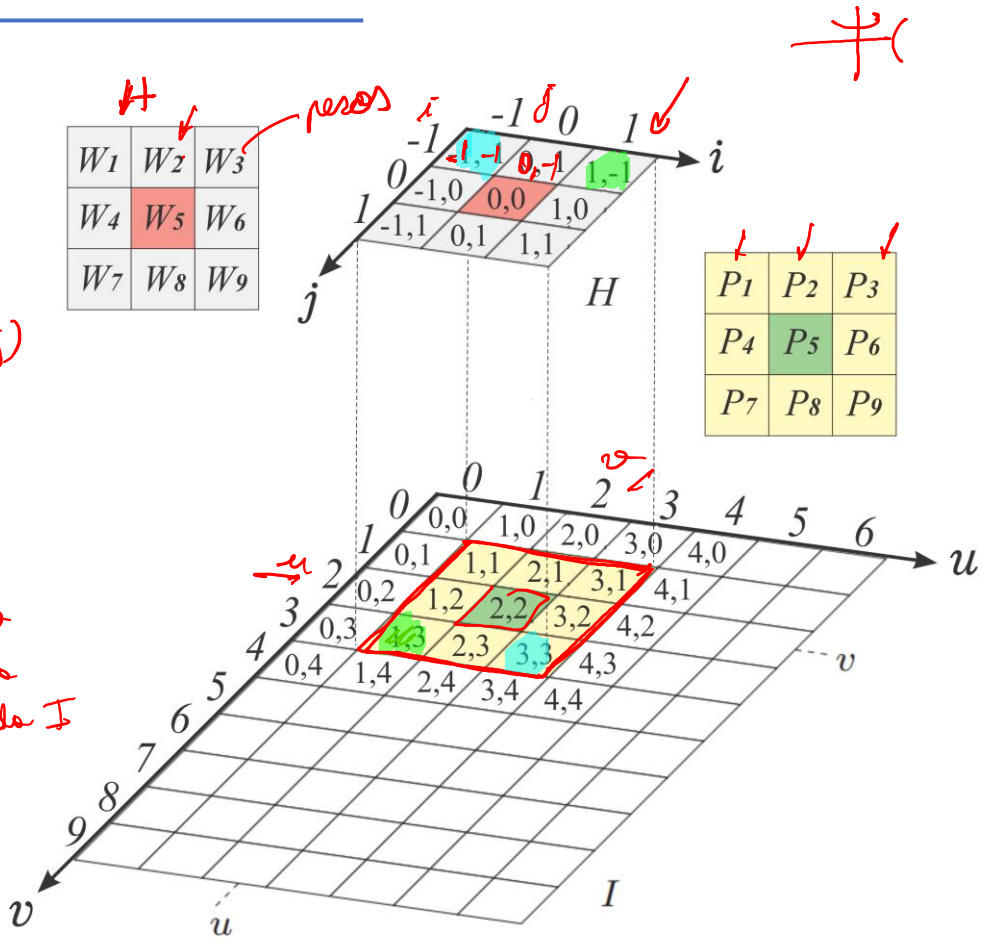


Figura: adaptado de Wilhelm Burger and Mark J. Burge. Digital Image Processing - An Algorithmic Introduction. 3rd ed. Springer, 2008, Fig. 4.4

Neighborhood Processing: Filtro espacial

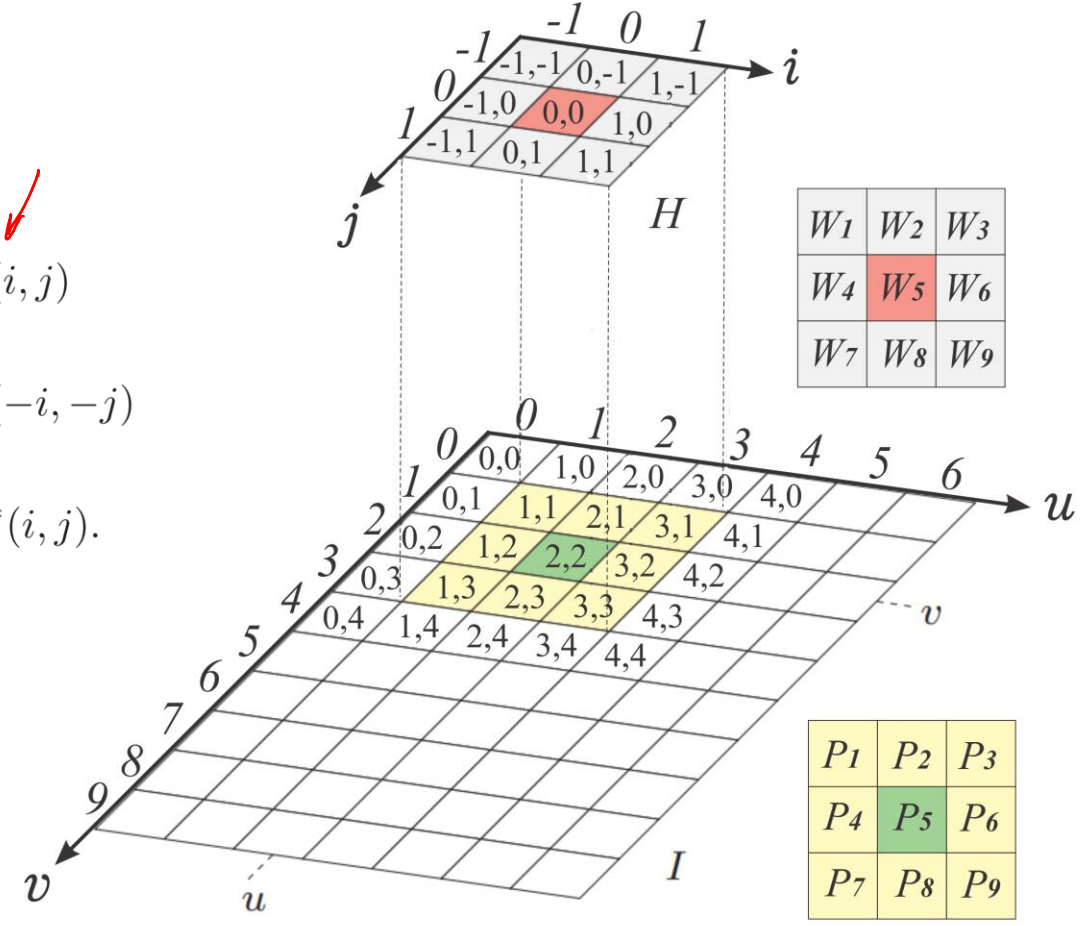
Convolução: passo-a-passo

$$I' = I * H$$

$$I'(u,v) = \sum_{(i,j)\in R_H} I(u-i, v-j) \cdot H(i,j)$$

$$= \sum_{(i,j)\in R_H} I(u+i, v+j) \cdot H(-i,-j)$$

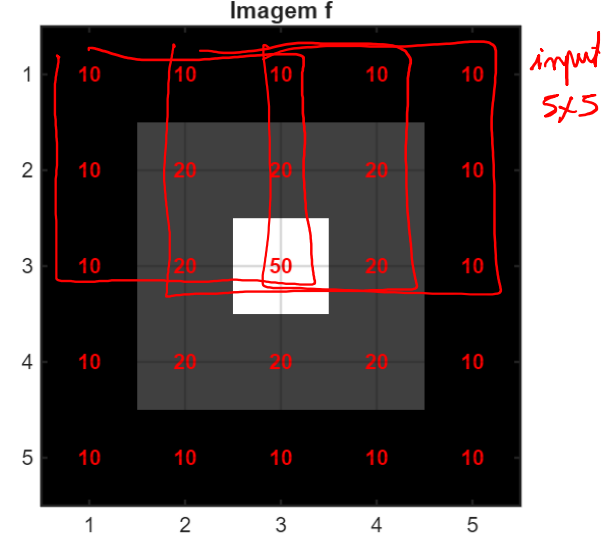
$$= \sum_{(i,j)\in R_H} I(u+i, v+j) \cdot H^*(i,j).$$



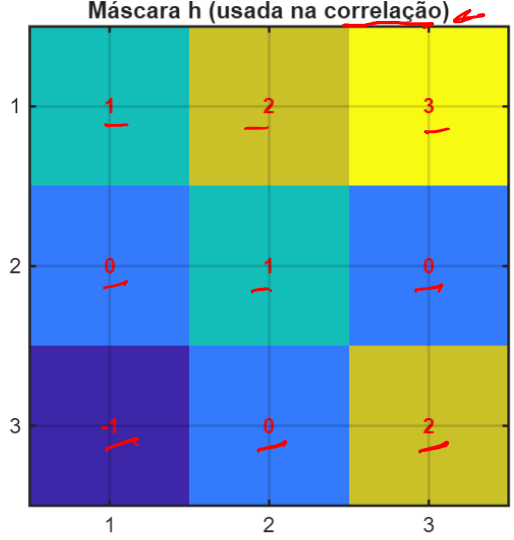
MATLAB: Correlação e Convolução

MATLAB usa correlação por padrão

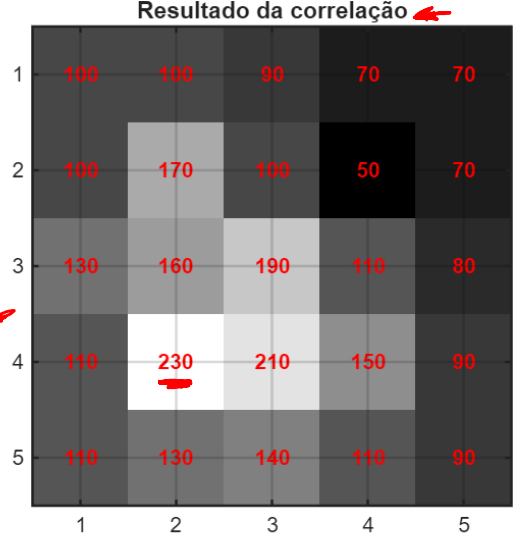
Operação	Código MATLAB	Máscara
Correlação	imfilter(f,h); ou imfilter(f,h,'cor');	Não inverte
Convolução	imfilter(f,h,'conv');	inverte



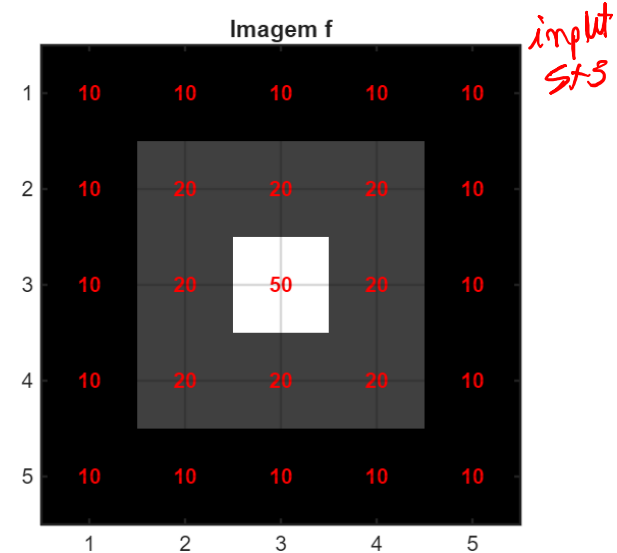
input 5x5



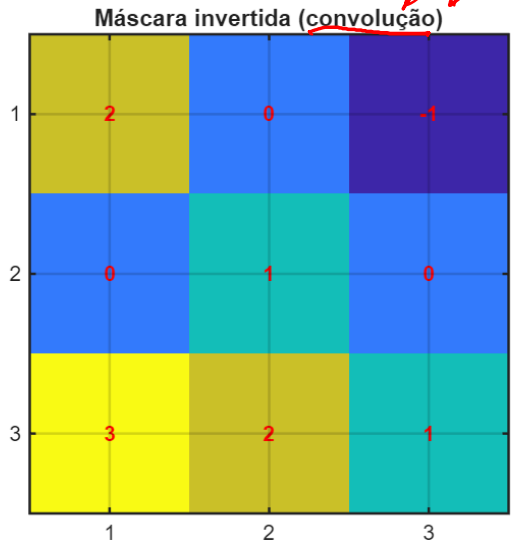
3x3

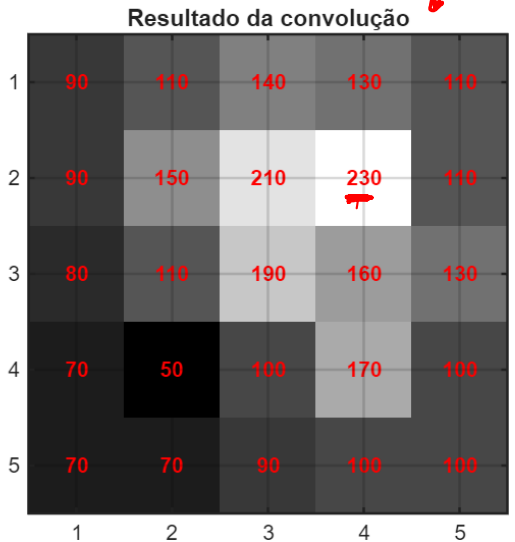


=



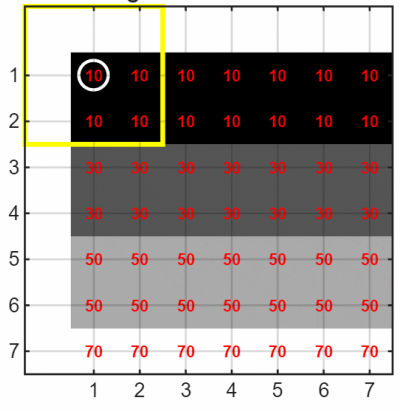
input 5x5

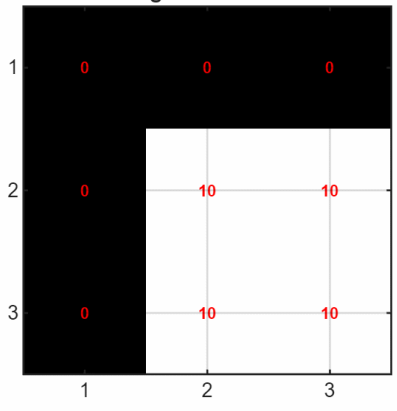


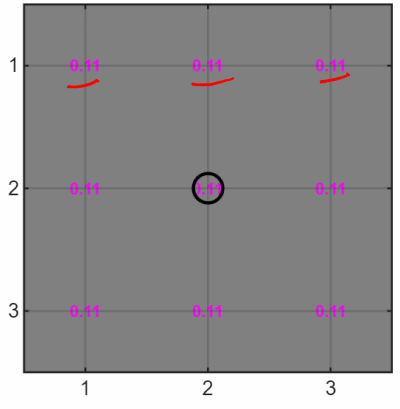


Neighborhood Processing: Filtro espacial

Correlação





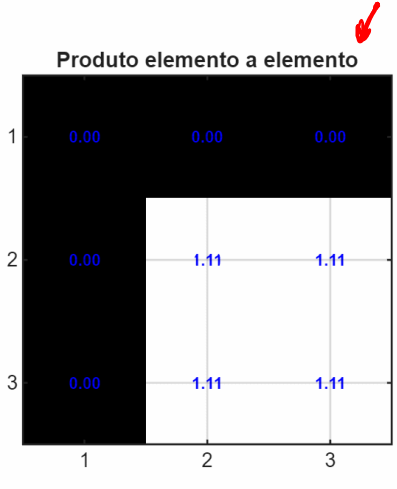


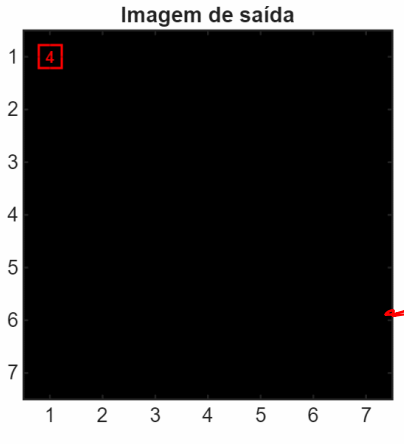
Posição atual: (1,1)

Cálculo:
G(1,1) = soma(R .* H)
G(1,1) = 4.44

O pixel de saída corresponde ao centro da janela 3x3.

O círculo marca o centro da região e da máscara.





Filtros lineares

Aditividade: $T(I_1 + I_2) = T(I_1) + T(I_2)$ Homogeneidade: $T(a \cdot I) = a \cdot T(I)$

Filtro passa-baixas (*image smoothing*)

- Atenua componentes de alta-frequência da imagem
  - Componentes de alta frequência são variações significativas nas intensidades de pixels próximos ⇒ isto é, são descontinuidades ⇒ isto é, são os detalhes finos da imagem ⇒ isto é, ruído e bordas.
- ***image smoothing*** (suavização de imagem): suavização da imagem ⇒ redução de ruídos e detalhes da imagem
- Vamos usar: **filtro da média (box-filter)** e **Gaussiano**

Neighborhood Processing: Filtro espacial

Filtros lineares (filho da média ou box-filter)

$$h = \frac{1}{N^2}$$

- Todos os pesos são iguais
- Simples
- Introduz borramento artificial

Box filter 3 x 3

$$h = \frac{1}{9} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

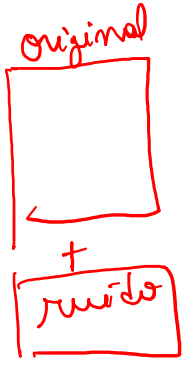
9

Imagem 3 x 3

$$I = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 1 \end{bmatrix}$$

pixel claro em uma região escura

Filtro da média (box filter) (passa-baixas, suavização)



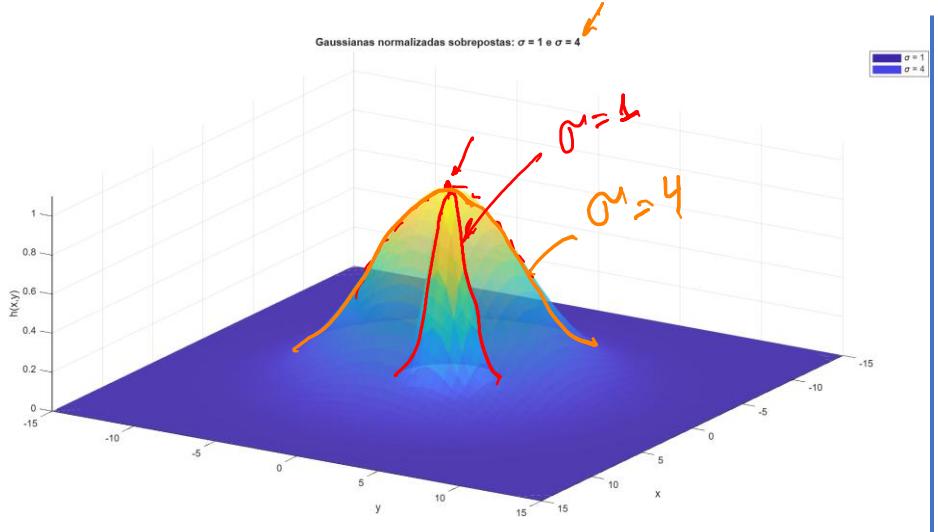


Abrir durante a aula: https://youtu.be/KuXjwB4LzSA?t=548

Neighborhood Processing: Filtro Gaussiano (passa-baixas, suavização)

Mais usado que o box-filter porque:

- isotrópico;
- atribui maior peso ao pixel central
- atribui pesos progressivamente menores aos pixels mais distantes do centro do kernel;
- produz uma ponderação espacial suave da vizinhança, minimizando descontinuidades.



$$h(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$g(x,y) = f(x,y) * h(x,y)$$

Ex_GaussianasEfeitoSigma3D.m

O Filtro Gaussiano em PDI faz parte dos chamados filtros lineares.

Como o kernel gaussiano é fixo, a operação satisfaz as propriedades de ADITIVIDADE E HOMOGENEIDADE

O filtro gaussiano é classificado como:

- Linear: satisfaz aditividade e homogeneidade;
- linear e invariante ao deslocamento (LSI – Linear and Shift-Invariant), pois o mesmo kernel fixo é aplicado em toda a imagem
- passa-baixas;
- filtro de suavização;

O parâmetro σ controla o grau de suavização: quanto maior σ, mais larga é a Gausiana e maior o borramento.

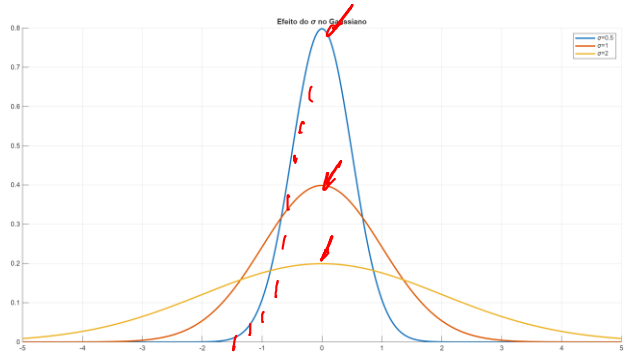
Neighborhood Processing: Filtro Gaussiano - kernel

Normalização:

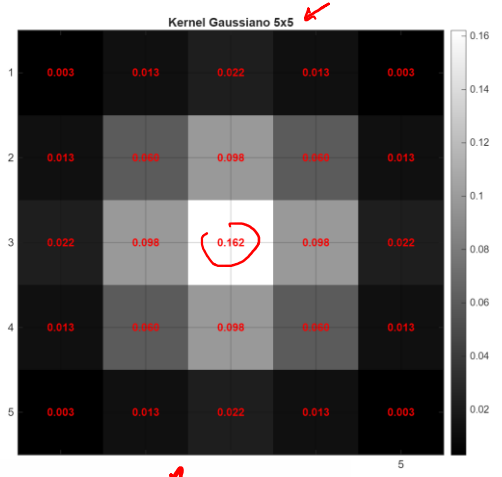
$$g(x,y) = \sum_m \sum_n h(m,n) f(x-m, y-n)$$

$$h(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$h_{norm}(i,j) = \frac{h(i,j)}{\sum_{i,j} h(i,j)} \qquad \sum_i \sum_j h_{norm}(i,j) = 1$$



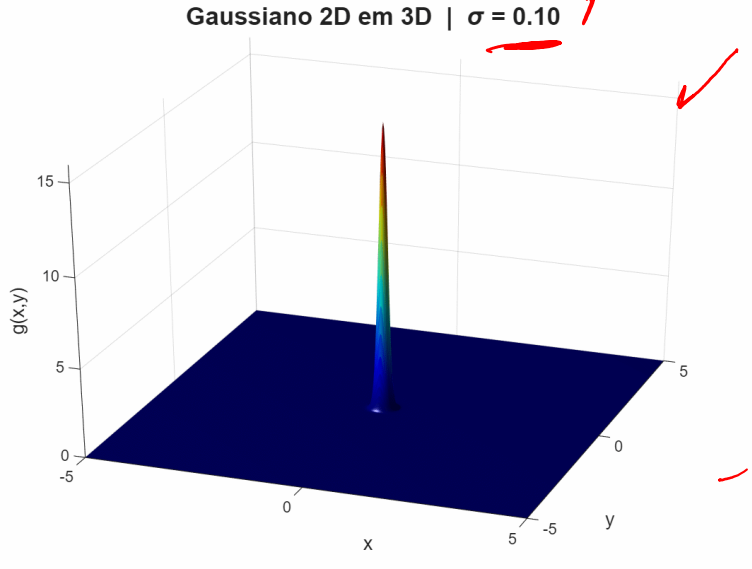
Kernel 5x5 para $\sigma = 1$



A normalização garante ganho DC unitário, evitando alteração do nível médio em regiões de intensidade constante, por ex., se todos os pixels da vizinhança tiverem a intensidade 100, a saída também será 100.

MATLAB:

```
h = exp(-(x.^2 + y.^2)/(2*sigma^2));
h = h / sum(h(:));
```



Neighborhood Processing: Filtro Gaussiano - kernel

Regra prática para definir o tamanho aproximado do kernel (não é uma regra obrigatória):

$\uparrow \sigma$ → Gaussiana mais larga → pesos relevantes distribuídos por uma vizinhança maior → maior suavização

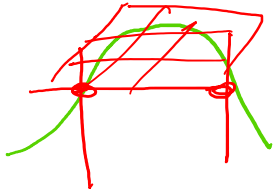
$\uparrow \sigma$ → pesos significativos mais distantes do centro → kernel maior para evitar truncamento excessivo

Regra prática $\pm 3\sigma$: $N \approx 2\lceil 3\sigma \rceil + 1$

onde $\lceil \cdot \rceil$ representa a função teto (ceiling), isto é, arredondamento para o inteiro imediatamente superior

No MATLAB

$$\mathrm{N} = 2 * \mathrm{ceil}(3 * \mathrm{sigma}) + 1$$



σ	tamanho do kernel $(\pm 3\sigma)$
0,5	5×5
1	7×7
2	13×13
4	25×25

Neighborhood Processing: MATLAB Filtro Gaussiano -kernel

Sintaxe para criar o filtro:

h = fspecial('gaussian', [M N], sigma);

'gaussian' → tipo de filtro

[M N] → tamanho do kernel

sigma → largura da Gaussiana

h → kernel normalizado $\sum h(i,j) = 1$

Sintaxe para aplicar o filtro:

g = imfilter(f, h, 'replicate');

Outra opção:

g = imgaussfilt(f, sigma, 'Padding', 'replicate');

Aplica o filtro sem necessidade de explicitar o kernel

IMPORTANTE: imgaussfilt escole automaticamente o tamanho do kernel usando $FilterSize = 2\lceil 2\sigma \rceil + 1$, em vez de $\pm 3\sigma$.

$\pm 2\sigma$: aproximação mais compacta, menor custo computacional

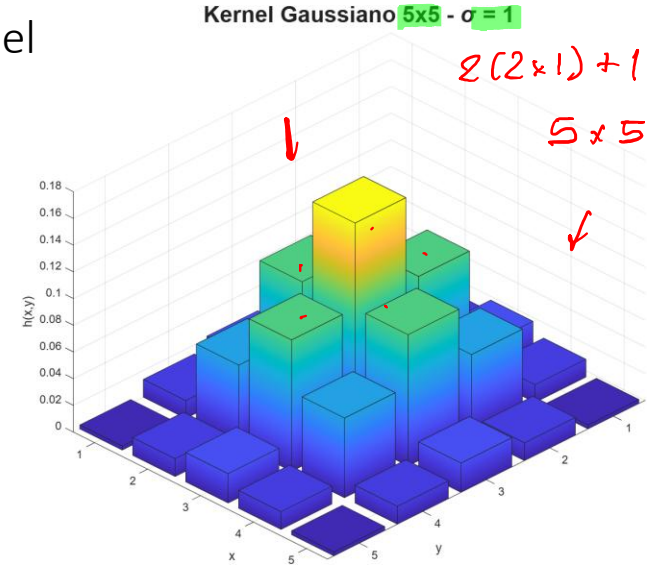
$\pm 3\sigma$: aproximação mais fiel da Gaussiana, com kernel maior

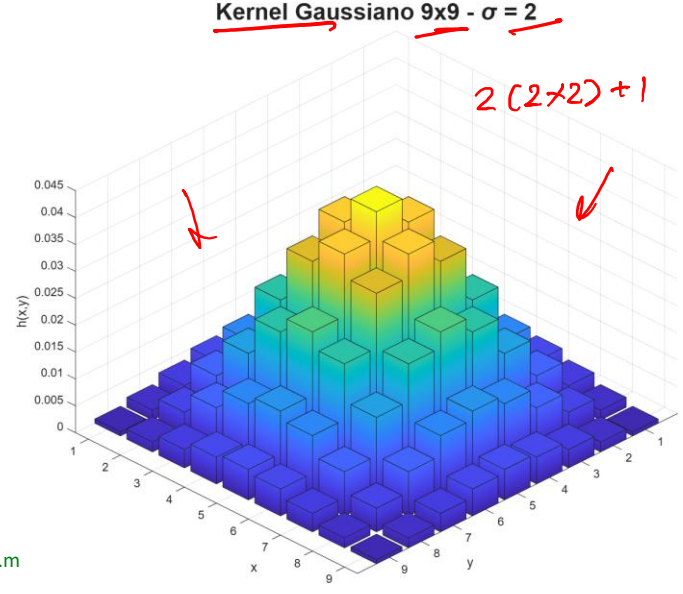
Opões de ajuste:

g = imgaussfilt(f, sigma, 'Padding', 'symmetric');

g = imgaussfilt(f, sigma, 'Padding', 'circular');

g = imgaussfilt(f, sigma, 'Padding', 0);



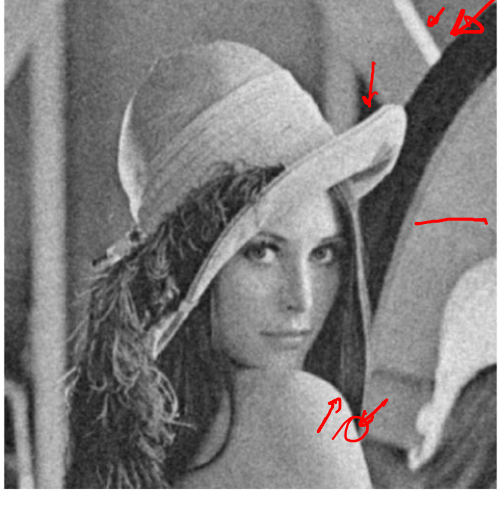


ig_Gauss3DEmBarrasColoridas.m

Neighborhood Processing: Efeito do Filtro Gaussiano em uma imagem










```
% Ruído Gaussiano:
% média = 0
% variância = 0.05
Iruido = imnoise(I, 'gaussian', 0, 0.05);
```

```
sigma1 = 1;
G1 = imgaussfilt(Iruido, sigma1, ...
'FilterSize', 5, ...
'Padding', 'replicate');
```

```
sigma2 = 2;
G2 = imgaussfilt(Iruido, sigma2, ...
'FilterSize', 9, ...
'Padding', 'replicate');
```

Obs: no imgassfilt, não é necessário especificar FilterSize. O MATLAB escolhe o tamanho do kernel a partir do σ.

LenaRuidoGaussiano.m

Neighborhood Processing: **Filtros lineares/espaciais: filtro Laplaciano**

Filtro Laplaciano

- Operador de realce de altas frequências;
- operador linear baseado na segunda derivada espacial;
- enfatiza variações rápidas de intensidade entre pixels vizinhos;
- Realça bordas e detalhes finos, mas também amplifica ruído;
- a saída do Laplaciano se parece com um mapa de bordas.

Operador de segunda derivada;

$$\nabla^2 f(x,y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

image sharpening combina o Lapaciano com a imagem original

$$g(x,y) = f(x,y) - c \cdot \nabla^2 f(x,y)$$

$c > 0$: controla a intensidade do realce

Exemplos de kernels discretos

A soma dos coeficientes é zero: resposta nula em regiões constantes.

4-vizinhança

0	1	0
1	-4	1
0	1	0

8-vizinhança

1	1	1
1	-8	1
1	1	1

Neighborhood Processing: filtro Laplaciano (filtro linear)

O Laplaciano enfatiza variações rápidas de intensidade, produzindo uma imagem semelhante a um mapa de bordas.

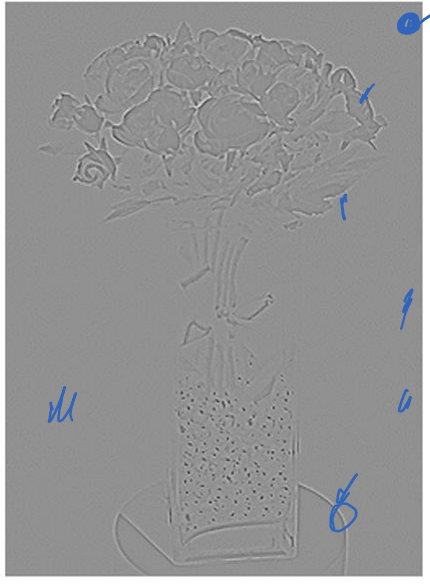


*

4-vizinhança

0	1	0
1	**-4**	1
0	1	0

=



$$g = \nabla^2 f$$

A soma dos coeficientes = 0

Resposta à componente DC = 0

Em regiões constante: $\nabla^2 f = 0$

o cinza intermediário representa aproximadamente zero, e tons claros/escuros representam sinais opostos da segunda derivada

Para realizar sharpening, o Laplaciano é combinado com a imagem original:

$$g = f - c \cdot \nabla^2 f, \qquad c > 0$$

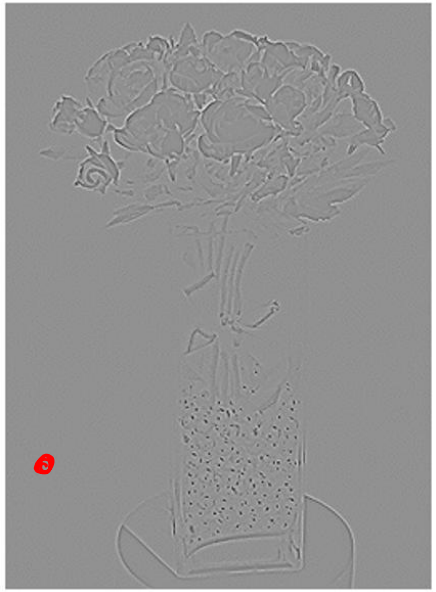
Neighborhood Processing: filtro Laplaciano (filtro linear)

Sharpening com filtro Laplaciano



$f(x,y)$

+



Visualização reescalada de $\nabla^2 f(x,y)$

O operador resulta em valores positivos e negativos. Por isso é necessário reescalar para exibir $\nabla^2 f(x,y)$

=



$g(x,y) = f(x,y) - c \cdot \nabla^2 f(x,y)$

$c = 3$

c: controla a intensidade do realce

mat2gray reescala toda a faixa dinâmica da imagem. Para comparar o sharpening com a imagem original, o clipping preserva melhor a escala de intensidades.

```
% rescalonamento
g = im2double(f) - c*lap;
g_vis = mat2gray(g);
imshow(g_vis);
```

```
% clipping
g(g < 0) = 0;
g(g > 1) = 1;
imshow(g);
```

```
% clipping
g = min(max(g,0),1);
imshow(g);
```

Obs.: o resultado do sharpening pode extrapolar a faixa de intensidade da imagem e requerer clipping ou reescalonamento para visualização.

Neighborhood Processing: **Unsharp masking** (Filtro Linear)

A ideia é:

- suavizar a imagem original;
- subtrair a versão suavizada da original;
- essa diferença contém principalmente detalhes de alta frequência;
- adicionar essa máscara de detalhes de volta à imagem original.

$f_{LP}(x,y)$ → imagem suavizada

$m(x,y) = f(x,y) - f_{LP}(x,y)$

$m(x,y)$ → é a unsharp mask

$$g(x,y) = f(x,y) + k \cdot m(x,y)$$

$$g(x,y) = f(x,y) + k \cdot [f(x,y) - f_{LP}(x,y)] \text{ com } k > 0$$

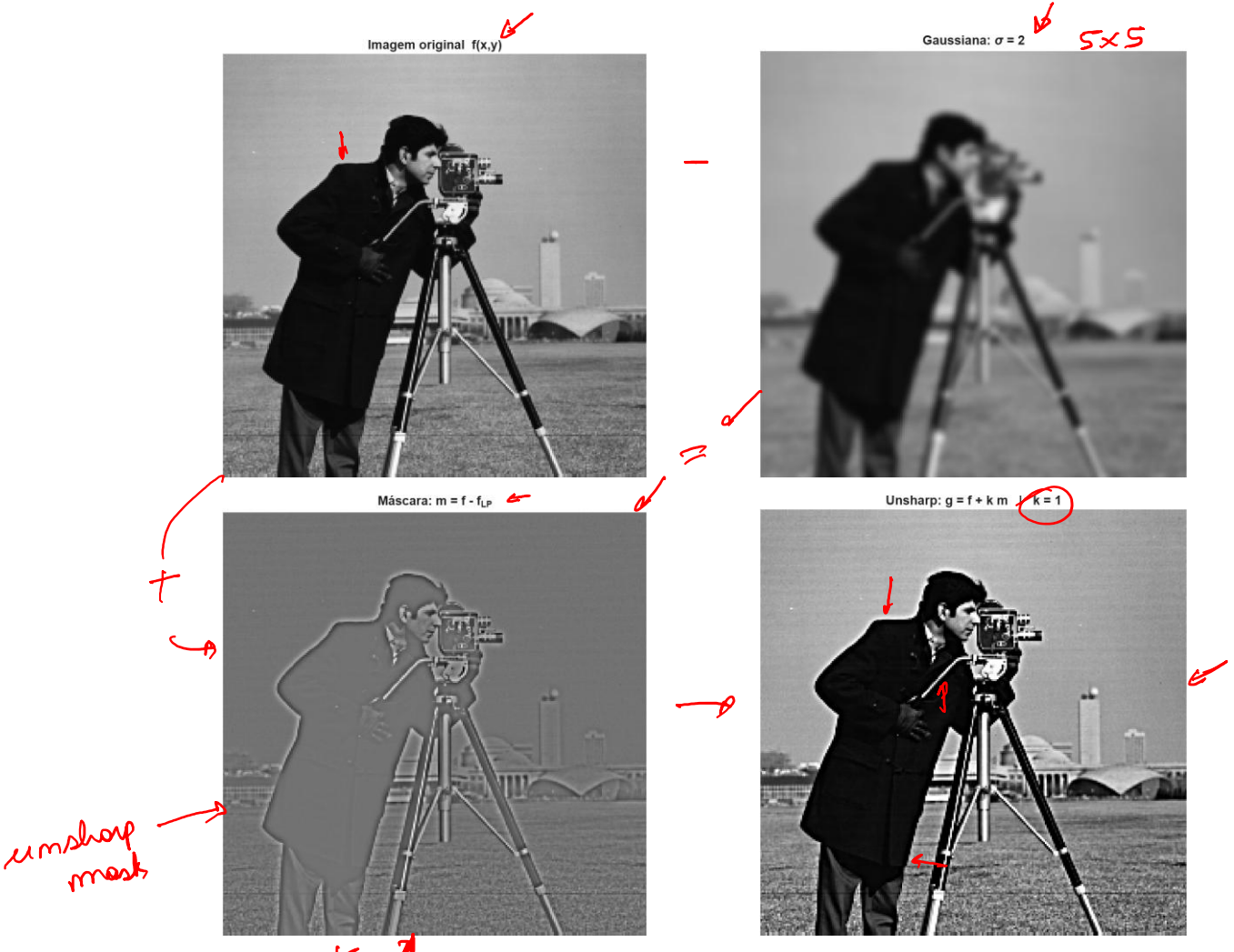
Laplaciano: extrai altas frequências via derivada → alta frequência obtida por segunda derivada; mais sensível a ruído.

Unsharp mask: extrai altas frequências via $f(x,y) - f_{LP}(x,y)$ → alta frequência obtida por $f - G_\sigma * f$; geralmente mais suave e controlável. σ da Gaussiana controla a escala dos detalhes que entram na máscara.

```
f = im2double(imread('imagem.png'));
sigma = 2;
fLP = imgaussfilt(f, sigma);
mask = f - fLP;
k = 1;
g = f + k*mask;
g = min(max(g,0),1);
imshow(g);
```

```
g = imsharpen(f);
```

Neighborhood Processing: **Unsharp masking** (Filtro Linear)



Ex_UnsharpMask.m

Neighborhood Processing: **Unsharp masking** (Filtro Linear)

k = 0.5



k = 1



k = 2



Neighborhood Processing: filtro da **mediana** - não usam convolução (não é um filtro linear)

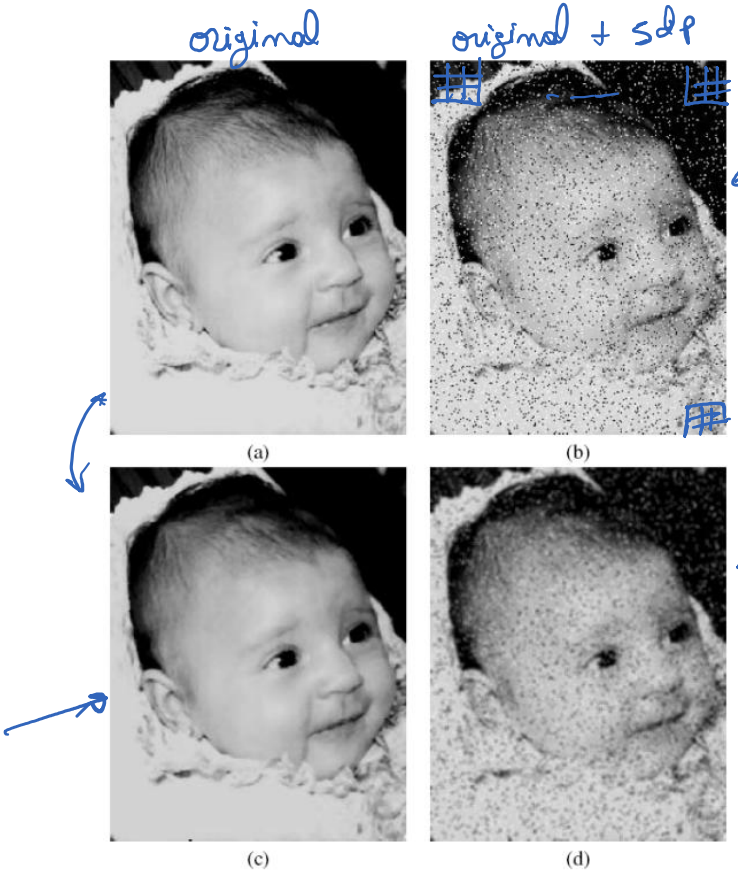
Aplicação típica: restauração de imagens com ruído do tipo sal-e-pimenta (salt-and-pepper noise).

10	10	10	10	10	10	10
10	10	10	10	10	10	10
30	30	255	30	30	30	30
30	30	30	0	30	30	30
50	50	0	50	50	50	50
50	50	50	50	50	50	50
70	70	70	70	70	70	70

0 10 10 10 30 30 30 30 255

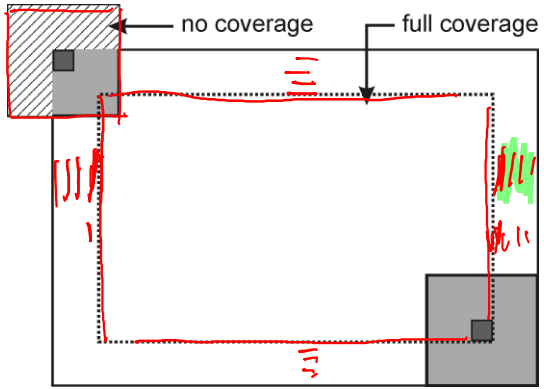
Neighborhood Processing: filtro da mediana (filtro linear)

Aplicação típica: remoção ruído salt & pepper



Oge Marques, Practical Image and Video Processing Using MATLAB, John Wiley and Sons, 2011, Fig. 10.9.

Neighborhood Processing: Tratamento das bordas da imagem



Boundary Options

Option	Description
Padding Options	
numeric scalar, X	Input array values outside the bounds of the array are assigned the value X. When no padding option is specified, the default is `0`.
`'symmetric'`	Input array values outside the bounds of the array are computed by mirror-reflecting the array across the array border.
`'replicate'`	Input array values outside the bounds of the array are assumed to equal the nearest array border value.
`'circular'`	Input array values outside the bounds of the array are computed by implicitly assuming the input array is periodic.
Output Size	
`'same'`	The output array is the same size as the input array. This is the default behavior when no output size options are specified.
`'full'`	The output array is the full filtered result, and so is larger than the input array.
Correlation and Convolution Options	
`'corr'`	`imfilter` performs multidimensional filtering using correlation, which is the same way that `filter2` performs filtering. When no correlation or convolution option is specified, `imfilter` uses correlation.
`'conv'`	`imfilter` performs multidimensional filtering using convolution.

https://www.mathworks.com/help/images/ref/imfilter.html

Figura: Oge Marques, Practical Image and Video Processing Using MATLAB, John Wiley and Sons, 2011, Fig. 10.4

Neighborhood Processing: Image Shapening with Composite Laplacian

Convolução é uma operação linear, logo podemos fazer a seguinte operação em uma imagem I:

$$I * H_1 + I * H_2 = I * (H_1 + H_2)$$

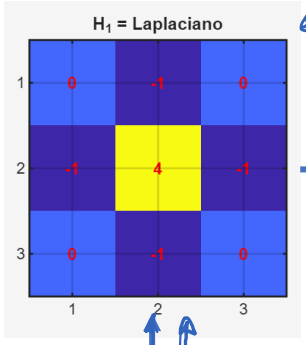
$$H_1 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \qquad H_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$H = H_1 + H_2 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Sharpening = Original Image + Edges

Output = [Image + Edges] = $I * H$

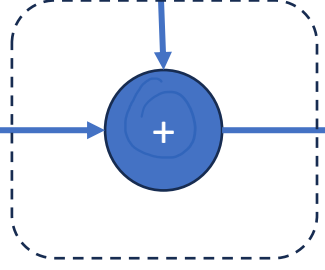
Neighborhood Processing: Image Shapening with Composite Laplacian



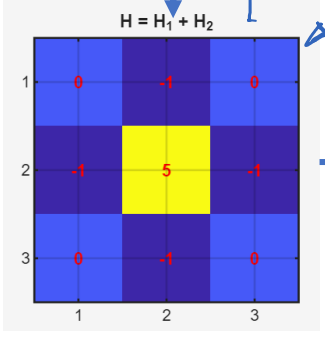



```
I_lap = imfilter(I, H1, 'replicate');
I_sharp = imfilter(I, H, 'replicate');
```









compositive laplacian

Ex_CompositiveLaplacian.m